BrandStore Phase 2, Task 1: API Contract Review and Application Plan

Item	Value
Author	Ahmed
Date	2026-11-12
Branch	`feature/api-contract-and-app-plan` (convention in section 8)
Spec	`api/openapi.json` (OpenAPI 3.0.3)
Mock server	`mock-server/server.mjs` (`npm run mock`, port 4010; demo login `demo@brandstore.test` / `Passw0rd!`)
Evidence	`docs/api/ENDPOINTS.md`, `docs/api/observed.json`, `docs/api/brandstore.postman_collection.json` (all generated by `npm run contract`)

1. Summary

- Exercised all 8 operations with 45 requests. Every status code in the spec (200, 201, 400, 401, 404, 429) was observed on the operations that document it, no undocumented code was observed, and every response body matched its schema: **0 problems**.
- Designed the `src/` module layout (`api/`, `components/`, `state/`, `utils/`, 31 files) and enforced its boundaries with ESLint.
- Chose where each piece of data is stored by what must survive refresh, tab close and logout.
- Added `package.json` scripts (dev, lint, format, test, contract), shared ESLint and Prettier configuration, CI, and a test plan led by cart math, auth and query-string building.
- `npm run lint` and `npm run format` (and `format:check`) pass on a clean checkout of this branch.

2. Endpoint Table (spec vs observed)

Copied from `docs/api/ENDPOINTS.md` (generated by `npm run contract`).

Method	Path	Auth	Summary	Status codes in spec	Status codes observed	Error codes observed	Success JSON (top level; data fields)
GET	`/products`	none	List products with filters, sorting and pagination	200, 400, 429	200, 400, 429	VALIDATION_ERROR, RATE_LIMITED	{data: [id, name, category, price, compareAtPrice, currency, rating, reviewCount, stock, image], meta: {page: integer, limit: integer, total: integer, totalPages: integer}}
GET	`/products/{id}`	none	Get one product	200, 400, 404, 429	200, 400, 404, 429	NOT_FOUND, VALIDATION_ERROR, RATE_LIMITED	{data: id, name, category, price, compareAtPrice, currency, rating, reviewCount, stock, image, description, specs}
POST	`/auth/login`	none	Exchange email and password for a bearer token	200, 400, 401, 429	200, 400, 401, 429	VALIDATION_ERROR, INVALID_JSON, INVALID_CREDENTIALS, RATE_LIMITED, TOO_MANY_LOGIN_ATTEMPTS	{data: token, tokenType, expiresIn, user}
GET	`/account/orders`	Bearer	List the signed-in user's orders, newest first	200, 400, 401, 429	200, 400, 401, 429	UNAUTHENTICATED, INVALID_TOKEN, VALIDATION_ERROR, RATE_LIMITED	{data: [id, createdAt, status, items, subtotal, shipping, total, currency, cancellable, cancelledAt, cancelReason], meta: {page: integer, limit: integer, total: integer, totalPages: integer}}
PATCH	`/account/orders/{id}/cancel`	Bearer	Cancel an order that is still processing	200, 400, 401, 404, 429	200, 400, 401, 404, 429	UNAUTHENTICATED, VALIDATION_ERROR, ORDER_NOT_CANCELLABLE, NOT_FOUND, RATE_LIMITED	{data: id, createdAt, status, items, subtotal, shipping, total, currency, cancelledAt, cancelReason, cancellable}
GET	`/account/addresses`	Bearer	List saved addresses	200, 401, 429	200, 401, 429	UNAUTHENTICATED, RATE_LIMITED	{data: [id, label, fullName, line1, line2, city, postalCode, country, phone, isDefault]}
PUT	`/account/addresses/{id}`	Bearer	Create or replace the address with this client-chosen id (idempotent)	200, 201, 400, 401, 429	200, 201, 400, 401, 429	UNAUTHENTICATED, VALIDATION_ERROR, RATE_LIMITED	{data: id, label, fullName, line1, line2, city, postalCode, country, phone, isDefault}

Method	Path	Auth	Summary	Status codes in spec	Status codes observed	Error codes observed	Success JSON (top level; data fields)
DELETE	`/account/addresses/{id}`	Bearer	Delete an address	200, 401, 404, 429	200, 401, 404, 429	UNAUTHENTICATED, NOT_FOUND, RATE_LIMITED	{data: id, deleted}

Error body for every 4xx: `{ "error": { "code", "message", "details"?: [{ "field", "message" }] } }`. Every response carries `X-RateLimit-Limit` and `X-RateLimit-Remaining`; 429 adds `Retry-After`; 401 adds `WWW-Authenticate: Bearer`; 201 adds `Location`. Outside the 8 operations: unknown path -> 404, wrong method -> 405 with `Allow`, `OPTIONS` -> 204 (CORS).

How each status was produced (full log of 45 requests: `docs/api/ENDPOINTS.md`):

Status	Produced by
200	Valid requests; `GET /products?page=99` returns 200 with empty `data` (not 404); PUT on an existing address id
201	`PUT /account/addresses/office` when `office` did not exist (with `Location`)
400	`limit=500`, `minPrice>maxPrice`, unknown `category`/`sort`, bad product id, login without fields, malformed JSON, `status=lost`, cancel reason over 200 chars, cancelling a delivered or already cancelled order (`ORDER_NOT_CANCELLABLE`), invalid address fields
401	`/account/*` without a token (`UNAUTHENTICATED`) or with an unknown token (`INVALID_TOKEN`); wrong password (`INVALID_CREDENTIALS`)
404	Unknown product, unknown order `o-9999`, deleting an address twice
429	61st request within 60 s from one client on every operation (`RATE_LIMITED`); 6th failed login for one email within 60 s (`TOO_MANY_LOGIN_ATTEMPTS`)

3. JSON Shapes Observed

```
GET /products            { data: ProductSummary[], meta: { page, limit, total, totalPages } }
ProductSummary           { id, name, category, price (cents), compareAtPrice|null, currency, rating, reviewCount, stock, image: { url, alt } }
GET /products/{id}       { data: ProductSummary + { description, specs: [{ label, value }] } }
POST /auth/login         { data: { token, tokenType: "Bearer", expiresIn: 3600, user: { id, name, email } } }
GET /account/orders      { data: Order[], meta }
Order                    { id, createdAt, status, items: [{ productId, name, quantity, unitPrice, lineTotal }], subtotal, shipping, total, currency, cancellable,
cancelledAt|null, cancelReason|null }
PATCH .../cancel         { data: Order }   (status "cancelled", cancellable false)
GET /account/addresses   { data: Address[] }
Address                  { id, label, fullName, line1, line2|null, city, postalCode, country, phone|null, isDefault }
PUT /account/addresses/{id}     { data: Address }
DELETE /account/addresses/{id}  { data: { id, deleted: true } }
```

4. Module Layout

Folder	Owns	May import	Must not
`src/utils/`	Pure helpers: money in cents, cart math, query strings, validation, storage wrapper, DOM helper	nothing from the app	call `fetch`; read app state
`src/api/`	Talking to the REST API: one fetch wrapper (`http.js`), one file per resource, `ApiError`	`utils/`	touch the DOM or state; store anything
`src/state/`	What the app remembers: cart, saved items, session, catalog filters; persistence through `utils/storage.js`	`utils/`	call the API (results are passed in)
`src/components/`	Rendering and user events for one piece of UI; each exports `mount(root, deps)` and returns `unmount`	`api/`, `state/`, `utils/`	import `main.js` or reach outside its `root`
`src/main.js`	Composition root: creates the http client and stores, mounts components, connects 401 to sign-out	everything	contain business logic

Dependency direction: `main.js -> components -> (api, state) -> utils`. ESLint enforces it: restricted imports per folder, `fetch` only in `api/http.js`, Web Storage only in `utils/storage.js` (verified with probe files: 4 of 4 violations reported).

Full map, flows and the 31 files: `docs/ARCHITECTURE.md`. Every planned function exists as a documented stub (JSDoc signature, throws "planned but not implemented"), so boundaries are reviewable and lint-checked before coding begins.

5. Storage Strategy

Data	Where	Key	Refresh	Tab close	Logout	Why
Cart lines (`productId`, `quantity`, price snapshot)	localStorage	`brandstore:cart:v1`	kept	kept	kept	A shopper expects the cart tomorrow and after signing out/in. It holds no personal data. Prices are re-checked against `GET /products/{id}` before checkout, so a stale snapshot cannot be charged.
Save-for-later items	localStorage	`brandstore:saved:v1`	kept	kept	kept	A wishlist is only useful if it lasts; same content and risk level as the cart.
Session: token, `expiresAt`, user `{id, name, email}`	sessionStorage	`brandstore:session:v1`	kept	**cleared**	**cleared**	Refresh must not sign the user out, but a bearer token should not outlive the tab (shared computers). Cleared on logout and on any 401.
Transient flags: `returnTo` after login, "session expired" notice, dismissed rate-limit banner, last cancelled order id for a one-time message	sessionStorage	`brandstore:flags:v1`	kept	cleared	**cleared**	Only meaningful during this visit; must not leak into a later visit or another user's session.
Catalog filters, sort, page	URL query string	n/a	kept	n/a (in history and bookmarks)	unaffected	Shareable and back/forward work; storing them would make the URL lie.
Products, orders, addresses from the API	memory only	n/a	refetched	gone	cleared	The server is the source of truth. Orders and addresses are personal data and are never written to the device.

Reasons and trade-offs (why not localStorage for the token, XSS, cart after logout, versioned keys, private-mode failures): `docs/STORAGE.md`.

6. Tooling (package.json, ESLint, Prettier)

Script	Command	Verified here
`dev`	`vite`	No: Vite not installable in the build environment
`mock`	`node mock-server/server.mjs`	Yes
`contract`	`node scripts/contract.mjs`	Yes: 45 requests, 0 problems
`lint`	`eslint .` (ESLint 10.1.0)	**Yes: 0 problems, on a fresh clone**
`format` / `format:check`	`prettier --write .` / `prettier --check .` (Prettier 3.8.1)	**Yes: no changes / all files formatted, on a fresh clone**
`test`	`vitest run`	Test file run with a Vitest-compatible shim: 21 passed (see Deviations)
`test:e2e`	`playwright test`	No tests yet (planned)
`branch:check`	`node scripts/check-branch.mjs`	Yes
`check`	all of the above except e2e	Partly (as above)

Shared config: `eslint.config.mjs` (ESLint's recommended rules + `eqeqeq`, `prefer-const`, `no-var`

- the boundary rules), `.prettierrc.json`, `.prettierignore`, `.editorconfig`, CI in `.github/workflows/ci.yml`. Turning lint on found 6 real issues in the Phase 1 QA scripts (unused variables, `let` that should be `const`, a needless escape); all fixed and the Phase 1 checks re-run green.

7. Test Plan (summary)

Highest-risk module	Unit	Integration	E2E
Cart math (`utils/cart-math.js`, `state/cart.js`)	Yes: every function, table-driven, cents only	Yes: cart + storage persistence	Yes: add, change quantity, refresh, totals unchanged
Auth (`api/auth.js`, `api/http.js`, `state/session.js`)	Yes: expiry with fake clock; signOut keys	Yes (MSW): bearer header, 401 sign-out, 429 lockout	Yes: sign in, refresh, sign out
Query-string building (`utils/query-string.js`, `state/catalog.js`)	Yes: build/parse, encoding, defaults, round trip	Yes (MSW): exact request URL, 400 details	Yes: filter, paginate, Back, shared link

All other modules, the concrete cases and exit criteria: `docs/TEST_PLAN.md`.

8. Feature Branch and Naming Convention

Convention: `<type>/<short-description>`, type one of `feature | fix | docs | chore | test | refactor`, lowercase kebab-case, at most 50 characters, branched from `main`, merged by pull request with CI green. This work is on **`feature/api-contract-and-app-plan`**; `npm run branch:check` validates it. Details: `CONTRIBUTING.md`.

9. Deviations List

#	Deviation	Reason	Follow-up
1	`api/openapi.json` and `mock-server/` were written for this task	No OpenAPI file or mock server was provided with the task	When the official files arrive: `SPEC=<file> API_URL=<server> npm run contract` re-checks them, and this table is regenerated
2	Endpoints exercised by a script (Node `fetch`), not by clicking in a REST client	Repeatable evidence for every status code	`docs/api/brandstore.postman_collection.json` runs the same 36 requests in Postman or Insomnia
3	`PUT` and `DELETE` addresses take an id: `/account/addresses/{id}`	A collection-level `PUT`/`DELETE` would replace or wipe every address	Confirm with the official spec
4	Cancelling a non-cancellable order returns 400 (409 would be more conventional); `DELETE` returns 200 with a body, not 204	Stay inside the status codes listed for this task	Adjust if the official spec differs
5	405 (wrong method) and 204 (`OPTIONS`) exist beyond the listed codes	Correct HTTP behaviour; documented	none
6	No `package-lock.json`; Vite, Vitest, MSW and Playwright not installed here	The build environment had no package-registry access	Run `npm install`, commit the lock file, run `npm test` with real Vitest
7	ESLint's recommended rules are read from ESLint itself instead of the `@eslint/js` package	Same rule set, one dependency fewer, verifiable offline	none
8	Phase 1 files are linted but excluded from Prettier	Reformatting them would bury their history in a whitespace diff	Remove from `.prettierignore` as each page moves to `src/`
9	The Phase 1 Content-Security-Policy (`default-src 'self'`) would block calls to an API on another origin	Found while planning `api/http.js`	Add the API origin to `connect-src` when the API host is known

10. How to Verify

```
git checkout feature/api-contract-and-app-plan
npm install
npm run lint && npm run format:check && npm test && npm run contract
npm run mock     # then import docs/api/brandstore.postman_collection.json into Postman and Run
```